
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 14

Outros Tópicos sobre C

Exercícios (14.2–14.7)

Resolvidos passo a passo, com código completo

O capítulo final do núcleo de C

Contents

1	Exercício 14.2 – Produto com Argumentos Variáveis	2
2	Exercício 14.3 – Argumentos da Linha de Comando	4
3	Exercício 14.4 – Ordenar com -c ou -d	6
4	Exercício 14.5 – Arquivos Temporários	9
5	Exercício 14.6 – Tratamento de Sinais	11
6	Exercício 14.7 – Alocação Dinâmica com realloc	14

1. Exercício 14.2 – Produto com Argumentos Variáveis

Enunciado 14.2

Escreva um programa que calcule o produto de uma série de inteiros passados à função `product` por uma lista de argumentos de tamanhos variáveis. Teste com várias chamadas, cada uma com um número diferente de argumentos.

Resposta detalhada

```
1  /* Ex14.2: produto_variadic.c
2     Demonstra uso de stdarg.h para funcao variadica */
3  #include <stdio.h>
4  #include <stdarg.h>
5
6  /* O primeiro parametro fixo eh a quantidade de
7     inteiros que segue */
8  int product( int contagem, ... )
9  {
10     va_list args;
11     int i, prod = 1;
12
13     va_start( args, contagem );
14
15     for ( i = 0; i < contagem; ++i ) {
16         prod *= va_arg( args, int );
17     }
18
19     va_end( args ); /* SEMPRE chamar antes do return!
20                     */
21     return prod;
22 }
23
24 int main( void )
25 {
26     /* 2 argumentos */
27     printf( "Produto de 2 e 3:          %d\n",
28            product( 2, 2, 3 ) );
29
30     /* 3 argumentos */
31     printf( "Produto de 4, 5 e 6:      %d\n",
32            product( 3, 4, 5, 6 ) );
33
34     /* 5 argumentos */
35     printf( "Produto de 1, 2, 3, 4, 5: %d\n",
36            product( 5, 1, 2, 3, 4, 5 ) );
37
38     /* 1 argumento */
39     printf( "Produto de 7:              %d\n",
40            product( 1, 7 ) );
```

```
39
40     /* nenhum argumento variavel (so o contador) */
41     printf( "Produto vazio:          %d\n",
42            product( 0 ) );
43
44     return 0;
45 }
```

Saída:

Saída do programa

```
Produto de 2 e 3:  6
Produto de 4, 5 e 6: 120
Produto de 1, 2, 3, 4, 5: 120
Produto de 7:  7
Produto vazio:  1
```

Análise didática

Boa prática de programação

Anatomia de uma função variádica:

- Pelo menos um parâmetro nomeado** – aqui *contagem*. As reticências (...) sempre vêm *depois* de pelo menos um parâmetro fixo, que serve como “âncora”.
- Declarar `va_list`** – tipo opaco que representa o estado do percurso pelos argumentos.
- `va_start`** – inicializa, recebendo como segundo argumento o *último parâmetro nomeado* (não as reticências).
- Iterar com `va_arg`** – para cada argumento, especificamos o tipo esperado. Aqui sempre `int`.
- `va_end`** – liberar recursos internos. Não chamar pode causar comportamento indefinido.

Erro comum de programação

Armadilhas de funções variádicas:

- **Sem checagem de tipos:** `product(2, 3.14, 5)` compila mas causa estrago – `va_arg(args, int)` interpreta os bytes do `double` como `int`.
- **Promoções automáticas:** `char` e `short` são promovidos a `int`; `float` a `double`. Nunca use `va_arg(args, char)` ou `va_arg(args, float)` – comportamento indefinido.
- **Precisa de sentinela ou contador:** a função não sabe quantos argumentos foram passados. Por isso passamos *contagem* explicitamente. `printf` sabe pelo número de % no formato; outras funções usam um valor sentinela como `NULL` ou `-1`.

2. Exercício 14.3 – Argumentos da Linha de Comando

Enunciado 14.3

Escreva um programa que imprima os argumentos da linha de comando.

Resposta detalhada

```
1  /* Ex14.3: argumentos.c */
2  #include <stdio.h>
3
4  int main( int argc, char *argv[] )
5  {
6      int i;
7
8      printf( "Numero de argumentos (argc): %d\n\n",
9              argc );
10
11     printf( "Argumentos:\n" );
12     for ( i = 0; i < argc; ++i ) {
13         printf( "    argv[%d] = \"%s\"\n", i, argv[ i ]
14             );
15     }
16
17     return 0;
18 }
```

Compilação e execução:

```
1  $ gcc -o argumentos argumentos.c
2  $ ./argumentos ola mundo arquivo.txt 42
```

Saída:

Saída do programa

```
Numero de argumentos (argc):  5

Argumentos:
argv[0] = "./argumentos"
argv[1] = "ola"
argv[2] = "mundo"
argv[3] = "arquivo.txt"
argv[4] = "42"
```

Análise didática

Atenção

Pontos cruciais sobre argc/argv:

- `argv[0]` é o **nome do programa** (geralmente, com o caminho). Em alguns sistemas pode ser o caminho completo, em outros apenas o nome.
- **argc inclui o nome do programa.** Por isso `argc ≥ 1` sempre (em sistemas conformes).
- `argv[argc]` é **garantidamente NULL**. Você pode iterar até encontrar NULL em vez de usar `argc`.
- **Tudo é string.** Para usar `argv[1]` como número, converta com `atoi`, `atof` ou `strtol`.
- **O shell faz o “split”.** `./prog "ola mundo"` resulta em `argc = 2`, `argv[1] = "ola mundo"`. Sem aspas, seriam dois argumentos separados.

Forma alternativa equivalente:

```
1 int main( int argc, char **argv ) /* ponteiro para  
   ponteiro */
```

As duas formas (`char *argv[]` e `char **argv`) são intercambiáveis para parâmetros de função.

3. Exercício 14.4 – Ordenar com -c ou -d

Enunciado 14.4

Escreva um programa que classifique um array de inteiros em ordem crescente ou decrescente. O programa deve usar argumentos da linha de comando para passar -c (crescente) ou -d (decrescente).

Resposta detalhada

```
1  /* Ex14.4: ordena_cli.c
2     Uso:
3         ./ordena -c 5 3 8 1 4      (crescente)
4         ./ordena -d 5 3 8 1 4      (decrescente) */
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8
9  void bubbleSort( int *arr, int n, int crescente )
10 {
11     int i, j, temp;
12     for ( i = 0; i < n - 1; ++i ) {
13         for ( j = 0; j < n - 1 - i; ++j ) {
14             int trocar = crescente ? ( arr[ j ] > arr[
15                                     j + 1 ] )
16                                     : ( arr[ j ] < arr
17                                         [ j + 1 ] );
18
19             if ( trocar ) {
20                 temp      = arr[ j ];
21                 arr[ j ]   = arr[ j + 1 ];
22                 arr[ j + 1 ] = temp;
23             }
24         }
25     }
26 }
27
28 int main( int argc, char *argv[] )
29 {
30     int i, n, crescente;
31
32     if ( argc < 3 ) {
33         printf( "Uso: %s -c|-d num1 num2 ...\\n", argv[
34                 0 ] );
35         return 1;
36     }
37
38     /* Determina ordem */
39     if ( strcmp( argv[ 1 ], "-c" ) == 0 ) {
40         crescente = 1;
41     } else if ( strcmp( argv[ 1 ], "-d" ) == 0 ) {
```

```
38         crescente = 0;
39     } else {
40         printf( "Opcao invalida: %s. Use -c ou -d.\n",
41                 argv[ 1 ] );
42         return 1;
43     }
44
45     /* Quantidade de numeros */
46     n = argc - 2;
47
48     /* Aloca array dinamicamente -- conceito do Cap.
49        12 */
50     int *arr = malloc( n * sizeof( int ) );
51     if ( arr == NULL ) {
52         printf( "Erro: falta de memoria.\n" );
53         return 1;
54     }
55
56     /* Converte strings em inteiros */
57     for ( i = 0; i < n; ++i ) {
58         arr[ i ] = atoi( argv[ i + 2 ] );
59     }
60
61     /* Ordena */
62     bubbleSort( arr, n, crescente );
63
64     /* Exibe resultado */
65     printf( "Resultado (%s):\n",
66            crescente ? "crescente" : "decrecente" );
67     for ( i = 0; i < n; ++i ) {
68         printf( "%d ", arr[ i ] );
69     }
70     printf( "\n" );
71
72     free( arr );
73     return 0;
74 }
```

Exemplos de uso:

Saída do programa

```
$ ./ordena -c 5 3 8 1 4 9 2
Resultado (crescente):
1 2 3 4 5 8 9

$ ./ordena -d 5 3 8 1 4 9 2
Resultado (decrecente):
9 8 5 4 3 2 1
```


Análise didática

Boa prática de programação

Padrão Unix de opções: flags começam com - seguidas de letra (-c, -d, -v, etc.). Para múltiplas opções, há convenções:

- **Forma curta:** -c, -d, -v.
- **Forma longa:** -crescente, -decrecente, -verbose (GNU).
- **Combináveis:** -c -d -v equivale a -c -d -v.
- **Com valor:** -o saida.txt ou -output=saida.txt.

Para projetos sérios, use a biblioteca <unistd.h> com `getopt` (POSIX) ou `getopt_long` (GNU). Aqui implementamos manualmente para mostrar o conceito.

4. Exercício 14.5 – Arquivos Temporários

Enunciado 14.5

Escreva um programa que coloque um espaço entre cada caractere em um arquivo. Use um arquivo temporário para isso: primeiro grave o conteúdo modificado no temporário, depois copie de volta ao arquivo original.

Resposta detalhada

```
1  /* Ex14.5: espaca_chars.c
2     Insere espaco entre cada caractere de um arquivo */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int main( int argc, char *argv[] )
7  {
8     FILE *original, *temp;
9     int c;
10
11     if ( argc != 2 ) {
12         printf( "Uso: %s arquivo.txt\n", argv[ 0 ] );
13         return 1;
14     }
15
16     /* Abre o original para leitura */
17     if ( ( original = fopen( argv[ 1 ], "r" ) ) ==
18         NULL ) {
19         printf( "Erro: nao foi possivel abrir %s\n",
20             argv[ 1 ] );
21         return 1;
22     }
23
24     /* Cria arquivo temporario */
25     if ( ( temp = tmpfile() ) == NULL ) {
26         printf( "Erro: nao foi possivel criar arquivo
27             temporario.\n" );
28         fclose( original );
29         return 1;
30     }
31
32     /* Etapa 1: copia do original para o temporario
33        inserindo espacos */
34     while ( ( c = fgetc( original ) ) != EOF ) {
35         if ( c != '\n' && c != ' ' ) {
36             fputc( c, temp );
37             fputc( ' ', temp ); /* espaco apos cada
38                 caractere */
39         } else {
40             fputc( c, temp ); /* preserva quebras e
```

```

36         }
37     }
38     fclose( original );
39
40     /* Etapa 2: rebobina temporario e reabre original
41        para escrita */
42     rewind( temp );
43     if ( ( original = fopen( argv[ 1 ], "w" ) ) ==
44         NULL ) {
45         printf( "Erro: nao foi possivel reescrever %s\
46             n", argv[ 1 ] );
47         fclose( temp );
48         return 1;
49     }
50
51     /* Etapa 3: copia do temporario para o original */
52     while ( ( c = fgetc( temp ) ) != EOF ) {
53         fputc( c, original );
54     }
55
56     fclose( original );
57     fclose( temp ); /* tmpfile() eh deletado
58                     automaticamente */
59
60     printf( "Arquivo %s processado com sucesso.\n",
61         argv[ 1 ] );
62     return 0;
63 }

```

Exemplo:

Saída do programa

```

$ cat exemplo.txt
Ola mundo

$ ./espaca_chars exemplo.txt
Arquivo exemplo.txt processado com sucesso.

$ cat exemplo.txt
O l a m u n d o

```

Vantagens de tmpfile():

- Nome único garantido pelo sistema.
- **Arquivo deletado automaticamente** ao fechar ou ao terminar o programa.
- Sem risco de colisão de nomes com outros processos.
- Aberto em modo binário "wb+" (leitura e escrita).

5. Exercício 14.6 – Tratamento de Sinais

Enunciado 14.6

Escreva um programa com tratadores de sinal para SIGABRT e SIGINT. Teste chamando `abort` (gera SIGABRT) e digitando `Ctrl+C` (gera SIGINT).

Resposta detalhada

```
1  /* Ex14.6: sinais.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <signal.h>
5
6  void tratadorSinal( int sinalRecebido )
7  {
8      /* IMPORTANTE: re-registrar o handler porque
9       alguns sistemas
10      resetam para SIG_DFL apos receber o sinal
11      */
12      signal( sinalRecebido, tratadorSinal );
13
14      printf( "\n*** Sinal capturado: " );
15
16      switch ( sinalRecebido ) {
17          case SIGABRT:
18              printf( "SIGABRT (abort) ***\n" );
19              printf( "    Programa solicitou aborto.\n" );
20              break;
21          case SIGINT:
22              printf( "SIGINT (Ctrl+C) ***\n" );
23              printf( "    Interrupcao do usuario
24              detectada.\n" );
25              break;
26          default:
27              printf( "Sinal %d ***\n", sinalRecebido );
28      }
29
30      printf( "    Encerrando programa de forma
31      controlada.\n" );
32      exit( EXIT_SUCCESS );
33  }
34
35  int main( void )
36  {
37      int opcao;
38
39      /* Registra tratadores */
40      signal( SIGABRT, tratadorSinal );
```

```
37     signal( SIGINT,  tratadorSinal );
38
39     printf( "Programa de teste de sinais\n" );
40     printf( "  1 - gerar SIGABRT (via abort)\n" );
41     printf( "  2 - gerar SIGABRT (via raise)\n" );
42     printf( "  3 - gerar SIGINT (via raise)\n" );
43     printf( "  4 - loop infinito (use Ctrl+C para
        testar SIGINT)\n" );
44     printf( "Escolha: " );
45     scanf( "%d", &opcao );
46
47     switch ( opcao ) {
48         case 1: abort();                break;
49         case 2: raise( SIGABRT );       break;
50         case 3: raise( SIGINT );        break;
51         case 4:
52             printf( "Pressione Ctrl+C para interromper
                ...\n" );
53             for ( ;; ) ; /* loop infinito */
54             break;
55         default:
56             printf( "Opcao invalida.\n" );
57     }
58
59     return 0;
60 }
```

Exemplo de execução:

Saída do programa

```
$ ./sinais
Programa de teste de sinais
1 - gerar SIGABRT (via abort)
2 - gerar SIGABRT (via raise)
3 - gerar SIGINT (via raise)
4 - loop infinito
Escolha: 4
Pressione Ctrl+C para interromper...
^C
*** Sinal capturado:  SIGINT (Ctrl+C) ***
Interrupcao do usuario detectada.
Encerrando programa de forma controlada.
```

Sinais comuns em sistemas Unix-like

Resposta detalhada

Sinal	Significado	Causa típica
SIGABRT	Aborto	chamada a <code>abort()</code>
SIGFPE	Erro aritmético	divisão por zero
SIGILL	Instrução ilegal	corrupção de código
SIGINT	Interrupção	Ctrl+C no terminal
SIGSEGV	Violação de segmentação	ponteiro inválido
SIGTERM	Término solicitado	<code>kill <pid></code>

Atenção

Cuidados com tratadores de sinais:

- Tratadores devem ser **curtos e simples**. Idealmente, apenas setar uma flag global `volatile sig_atomic_t`.
- **Nem todas as funções são seguras** em tratadores. `printf` e `malloc` *não são* `async-signal-safe`; podem deadlock.
- **SIGKILL e SIGSTOP não podem ser interceptados**. São o “mecanismo de força bruta” do SO.

6. Exercício 14.7 – Alocação Dinâmica com realloc

Enunciado 14.7

Escreva um programa que aloque um array de inteiros dinamicamente, com tamanho fornecido pelo teclado. Leia os elementos, imprima-os. Em seguida, redimensione para a metade do tamanho original e imprima os elementos restantes.

Resposta detalhada

```
1  /* Ex14.7: realloc_demo.c
2     Demonstra malloc, realloc e free */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  void imprimirArray( const int *arr, int n, const char
7     *titulo )
8  {
9     int i;
10    printf( "%s (%d elementos):\n ", titulo, n );
11    for ( i = 0; i < n; ++i ) {
12        printf( "%d ", arr[ i ] );
13    }
14    printf( "\n" );
15
16  int main( void )
17  {
18     int tamanho, i;
19     int *arr;
20
21     printf( "Tamanho do array: " );
22     scanf( "%d", &tamanho );
23
24     if ( tamanho <= 0 ) {
25         printf( "Tamanho invalido.\n" );
26         return 1;
27     }
28
29     /* Aloca com calloc (zera tudo) */
30     arr = calloc( tamanho, sizeof( int ) );
31     if ( arr == NULL ) {
32         printf( "Erro: memoria insuficiente.\n" );
33         return 1;
34     }
35
36     /* Le valores */
37     printf( "Digite %d inteiros:\n", tamanho );
38     for ( i = 0; i < tamanho; ++i ) {
```

```
39         scanf( "%d", &arr[ i ] );
40     }
41
42     imprimirArray( arr, tamanho, "Array original" );
43
44     /* Realoca para metade do tamanho */
45     int novoTamanho = tamanho / 2;
46     int *temp = realloc( arr, novoTamanho * sizeof(
47         int ) );
48
49     if ( temp == NULL ) {
50         printf( "Erro ao realocar.\n" );
51         free( arr ); /* libera bloco original */
52         return 1;
53     }
54     arr = temp; /* atualiza ponteiro */
55
56     imprimirArray( arr, novoTamanho, "Array apos
57         realloc (metade)" );
58
59     free( arr );
60     return 0;
61 }
```

Exemplo de execução:

Saída do programa

```
Tamanho do array: 6
Digite 6 inteiros:
10 20 30 40 50 60
Array original (6 elementos):
10 20 30 40 50 60
Array apos realloc (metade) (3 elementos):
10 20 30
```

Anatomia de realloc

Boa prática de programação

Como funciona realloc:

```
1 void *realloc( void *ptr, size_t novoTamanho );
```

Casos possíveis:

- Tamanho menor:** mantém os bytes iniciais, libera o resto. Pode retornar o mesmo ponteiro.
- Tamanho maior, espaço contíguo disponível:** expande no lugar.
- Tamanho maior, sem espaço:** aloca novo bloco em outro lugar, *copia* os dados antigos, libera o original. **O ponteiro muda!**

d) **Falha:** retorna NULL, o ponteiro original **permanece válido**.
Por isso o idioma:

```
1 int *temp = realloc( arr, novoTam * sizeof( int ) );
2 if ( temp != NULL ) {
3     arr = temp;           /* sucesso */
4 } else {
5     /* falha: arr ainda eh valido, precisa de free
6        dele */
7     free( arr );
8     return 1;
9 }
```

Erro comum de programação

Erro classico:

```
1 arr = realloc( arr, novoTam ); /* PERIGOSO */
```

Se `realloc` falhar, `arr` vira NULL e perdemos *permanentemente* o ponteiro para a memória original – **memory leak**!

Reflexão Final – Encerrando o Núcleo de C

14 capítulos, uma jornada completa

Chegamos ao fim do núcleo de C. Olhando para trás, completamos uma travessia notável:

Capítulos 1–2 – Os primeiros passos: de zero a programas funcionais.

Capítulos 3–5 – Programação estruturada: controle de fluxo, funções, recursão.

Capítulos 6–8 – Dados compostos: arrays, ponteiros (o coração de C!), strings.

Capítulo 9 – E/S formatada: domínio completo de `printf/scanf`.

Capítulos 10–12 – Estruturas e persistência: `struct`, arquivos, listas, pilhas, filas, árvores.

Capítulo 13 – O pré-processador: a meta-linguagem do compilador.

Capítulo 14 – Outros tópicos: variádicas, linha de comando, sinais, `realloc`, `Makefile`.

O que você ganhou:

- Capacidade de escrever programas profissionais em C.
- Compreensão de baixo nível: memória, ponteiros, bits, arquivos.
- Vocabulário para entender qualquer código C existente.
- Base sólida para qualquer linguagem moderna – C++, Rust, Go, Java herdam grande parte da sintaxe e semântica.
- Pré-requisito para sistemas operacionais, compiladores, embarcados, drivers, jogos, alta performance.

Próximos passos sugeridos:

- Leia código de projetos open-source escritos em C (kernel Linux, Git, Redis, SQLite, FFmpeg).
- Estude *The C Programming Language* (K&R) – o livro original de Kernighan e Ritchie.
- Aprenda C++ ou Rust – evoluções diretas de C com segurança e abstração modernas.
- Estude algoritmos avançados: *Introduction to Algorithms* (CLRS), Sedgewick.
- Pratique em desafios: Project Euler, Advent of Code, sistemas de competição de programação.

Parabéns por chegar até aqui! Os 14 capítulos de Deitel & Deitel formam um currículo completo e rigoroso. Você está oficialmente pronto para o próximo nível.